TD4-5
Interface AHB - Assertions

Thomas Sarno

02/10/2026

1 Introduction

Il vous est demandé de rédiger un compte-rendu pour ce TD dans lequel vous ferez figurer toutes les remarques pertinentes qui vous semblent nécessaires à la bonne compréhension du problème posé et de la solution apportée. Bien évidement il s'agit d'un travail personnel.

Dans le TD4 nous allons monter d'un niveau dans la hiérarchie du système, notre AES est maintenant inclus dans une sur-couche de communication qui va lui permettre d'être connecté à d'autres éléments. Le but de ce TD est de vérifier ces nouvelles fonctionnalités. Pour cela nous allons ajouter à notre palette d'outils les assertions.

Ci dessous vous trouverez un diagramme bloc (Figure 1) de l'AES et de la couche de communication qui permet de le stimuler.

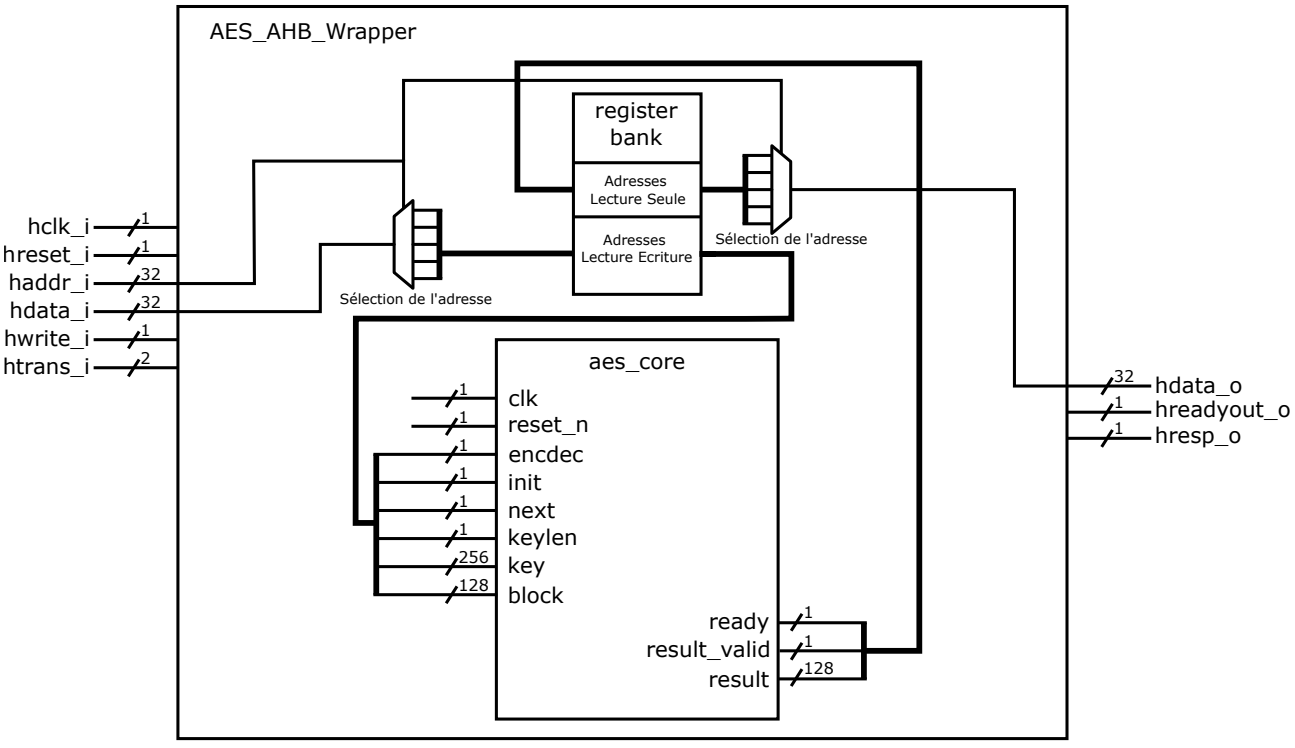


FIGURE 1 – Coeur AES et sur-couche de communication

2 Stimulus basiques

La première étape va être de créer les taches de bases pour lire et écrire dans la banque de registres qui est connectée à l'AES.

Pour cela vous pourrez vous servir des squelettes de `task` *simple_write* et *simple_read* pour reproduire le protocole, comme illustré dans les figures 2 et 3

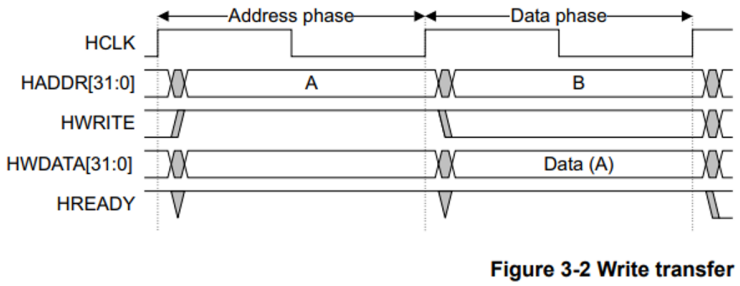


FIGURE 2 – Écriture AHB

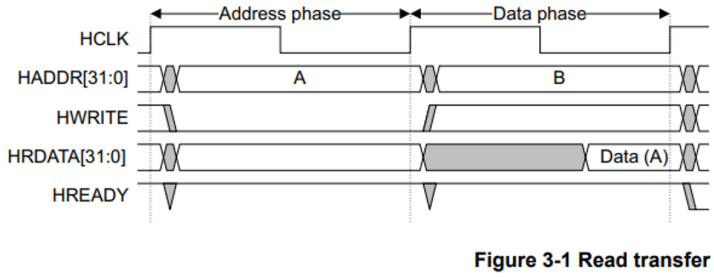


FIGURE 3 – Lecture AHB

Pour vérifier le bon fonctionnement de la commande d'écriture, écrivez la valeur 0xA5A5 à l'adresse 0x0002. Vous pourrez observer dans le signal `regbank_s` en utilisant l'introspection et confirmer qu'il contient la valeur que vous avez écrite à la bonne adresse.

Pour la lecture, vous allez lire l'adresse 0x0015 et de la même façon comparer la sortie *hdata_o* avec une observation directe de la valeur dans l'adresse correspondante du signal *regbank_s*.

3 Contrôle de l'AES

3.1 Paramètres et Données

Une fois que vous arrivez à effectuer les opérations de lecture et d'écriture, vous pouvez inclure ces `tasks` dans les `tasks` *setup_operation* et *send_data*.

Pour cela vous devez identifier les adresses de la banque de registres qui correspondent au entrées sorties que vous voulez utiliser. Ces informations se trouvent dans la documentation de l'AES.

Contrairement à ce que nous avons fait jusque là, vous ne pouvez pas mettre plusieurs signaux simultanément aux bonnes valeurs (sauf si ils sont à la même adresse). Il faut donc faire attention à l'ordre dans lequel vous allez envoyer les commandes.

Attention : certaines valeurs écrites dans la banque de registres ne doivent pas être conservées comme des états, mais interprétées comme des commandes instantanées. On parle ici des champs de type `W1P` (*Write-One-to-Pulse*) qui génèrent une action limitée à un seul cycle d'horloge lorsqu'on écrit la valeur 1.

Vous pouvez à ce stade verifier directement dans les waveforms que les commandes sont correctement interprétées et que les actions correspondantes sont générées. Pretez particulièrement attention aux signaux dont le comportement est une impulsion.

3.2 Attendre la fin des opérations

Vous noterez qu'en dehors du signal d'horloge, les commandes `@posedge` pour attendre les front montants de signaux ne fonctionnent plus. Vous devrez créer une nouvelle `task` pour cela.

Cette `task` que vous appellerez par exemple `wait_computation_end` devra lire périodiquement l'adresse `AES_STATUS_address` jusqu'à ce que le bit ready passe à 1. Cette adresse correspond aux sorties *ready* et *result_valid*, lorsque la valeur du bit correspondant passe à '1' vous pourrez continuer le déroulement des instructions.

3.3 Verification du fonctionnement de l'AES avec AHB

Après les opérations simple, vous devez maintenant vérifier que toutes les opérations fonctionnent. Cette étape est normalement anecdotique dans la mesure où vous avez déjà tous les outils en place pour un vérification complète.

4 Assertions Basiques

Les assertions visent à vérifier que certaines propriétés sont respectées dans le circuit. Ces constructions sont particulièrement utilisées pour les protocoles de communication qui peuvent avoir des constructions complexes. Notre cas reste relativement simple mais nous allons aussi pouvoir vérifier le bon fonctionnement et le cas échéant apporter des corrections.

Les assertions peuvent être utilisées à différents stades du flot de conception.
- l'implémentation
- la vérification
- l'intégration

4.1 Vérification des propriétés du protocole

L'objectif ici sera de vérifier les propriétés ci-dessous en utilisant des assertions. Il vous faudra donc définir ces propriétés avec des assertions et faire en sorte de les tester.

4.1.1 Propriété HTRANS_val

Le signal `htrans_i` est encodé sur 2 bits mais dans notre protocole simplifié, seulement les valeurs "00" et "10" sont valides. Si une autre valeur est utilisée il faut lever un *warning*.

4.1.2 Propriété HADDR_range

Seulement un sous-ensemble des adresses sont utilisées, si des adresses non-utilisées sont détectées il faut lever une erreur.

4.1.3 Propriété HADDR_access

Dans le même esprit certaine adresses n'ont pas d'accès en écriture depuis l'interface AHB. Si un requète en écriture est faite, il faut lever une erreur.

4.1.4 Propriété HTRANS_stable

Lorsque le signal `htrans_i` passe de IDLE (2'b00) à NONSEQ (2'b10) les signaux `haddr_i` et `hwrite_i` doivent rester statiques, dans le cas contraire il faut lever une erreur.

4.2 Propriétés du contrôle de l'AES

Pour que l'AES fonctionne, il y a aussi un protocole à respecter. Et ce protocole à des propriétés.

Proposez quatre propriétés pour l'AES ?

Ensuite vous écrirez les assertions correspondantes.

4.3 Sortie d'erreur et vérification des propriétés

Les assertions permettent de vérifier qu'un ensemble de propriétés est respecté par le circuit, en simulation ou lors d'une analyse formelle. Elles sont donc particulièrement utiles pour détecter des erreurs de protocole, de séquencement ou d'utilisation du bloc.

Dans un système réel, certaines violations doivent également être signalées à l'extérieur du bloc afin que le maître puisse réagir. C'est le rôle de la sortie `HRESP`, qui indique si une transaction AHB s'est déroulée correctement ou non.

On distinguera donc :

- Les propriétés vérifiées par assertions, qui servent à détecter les comportements non conformes ;
- Les erreurs de protocole ou d'accès, qui doivent en plus être signalées via `HRESP` lorsque cela est nécessaire.

On va supposer ici que le protocole AHB est correctement géré par le contrôleur de bus. En revanche, celui-ci ne connaît pas forcément les spécificités du bloc AES. C'est pourquoi certaines utilisations non supportées du bloc devront à la fois :

— déclencher une propriété en erreur ou en avertissement ;
— et, lorsque c'est pertinent, produire une réponse d'erreur via `HRESP`.

Vous prendrez l'exemple de l'erreur *forbidden address error* pour implémenter les erreurs de commande de l'AES que vous avez identifiées.

La sortie `HRESP` renvoie `OK` "1" lorsque tout va bien et `ERREUR` le cas échéant. La figure 4 vous indique comment se comporte les signaux `H_RESP` et `H_READY` lors d'une erreur.

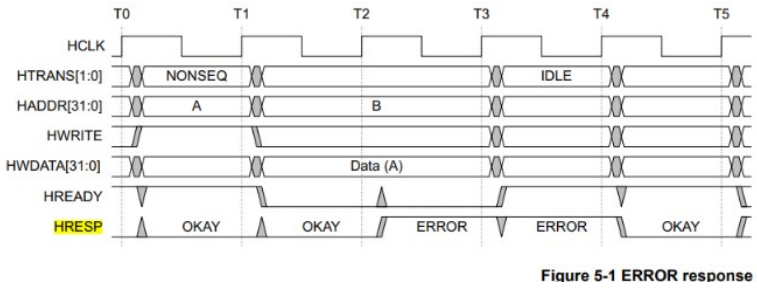


FIGURE 4 – Sortie H_RESP AHB